

PROYECTO INTEGRADOR · SISTEMAS OPERATIVOS

Diseño e Implementación de un Mini Monitor de Recursos para Linux Basado en Python

Danny Ayuquina · Saray Canarte · Domenica Villagomez

Carrera de Ingeniería de Software

El problema

Los sistemas operativos modernos exponen el estado de sus recursos a través de mecanismos como el sistema de archivos virtual `proc`, pero gran parte de esa lógica queda oculta detrás de herramientas de alto nivel. Esto dificulta que un estudiante de Ingeniería de Software compruebe, de forma directa y verificable, los conceptos centrales de la asignatura de Sistemas Operativos.

1

Abstracción excesiva

Bibliotecas como `psutil`, y herramientas como `htop` o `glances`, ocultan el acceso directo al kernel de Linux.

2

Conceptos dispersos

`proc`, `fork`, hilos y operaciones **CRUD rara vez se integran en un solo proyecto académico verificable.**

3

Falta de validación práctica

Sin comparar resultados contra utilidades estándar (`free`, `df`, `ps`, `who`) no hay evidencia objetiva de que la lectura del sistema sea correcta.

Objetivos

Objetivo general

Aplicar de forma práctica los conceptos fundamentales de Sistemas Operativos mediante el diseño e implementación de un monitor de recursos para Linux desarrollado en Python.

1

Lectura del sistema

Obtener el estado de CPU, memoria, disco y red directamente desde /proc y comandos estándar del sistema.

2

Concurrencia real

Combinar procesos (fork) e hilos (threading) para capturar el estado del sistema sin bloquear la interfaz.

3

Persistencia CRUD

Registrar un historial de capturas en SQLite con creación, consulta, actualización y eliminación.

4

Validación objetiva

Contrastar los resultados con utilidades estándar de Linux: free, df, ps y who.

Arquitectura

Presentación

Interfaz de texto interactiva (Textual), organizada en pestañas.

Servicios

Unifica el acceso a los datos del sistema para las capas superiores.

Concurrencia

Orquesta la captura mediante un proceso hijo (fork) y dos hilos internos.

Repositorio

Ejecuta las operaciones CRUD sobre la base de datos SQLite.

Sistema operativo

Lee archivos de /proc y ejecuta comandos del sistema vía subprocess.

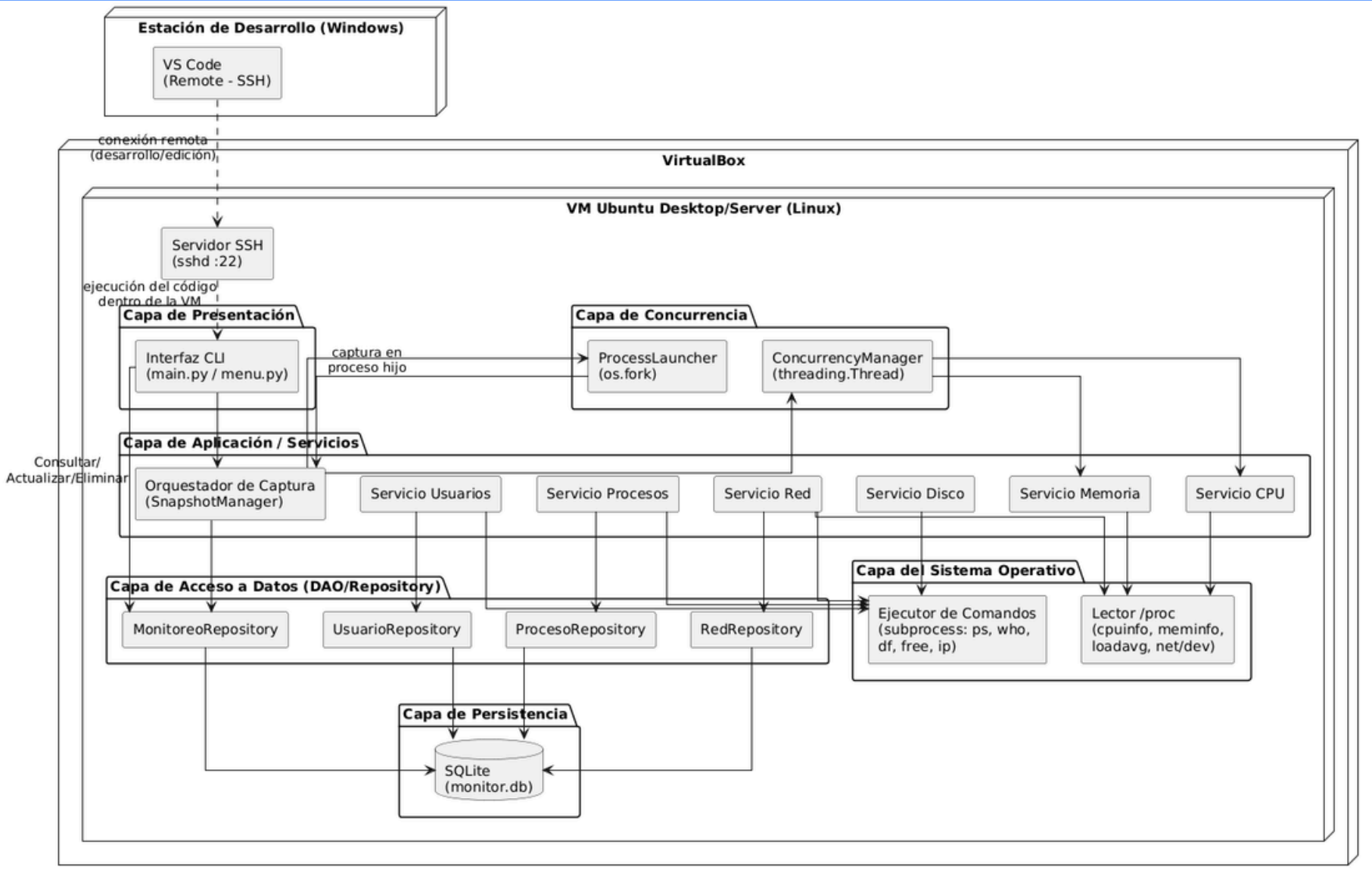


Fig. Arquitectura en capas del Mini Monitor de Recursos.

Tecnologías utilizadas



Python 3

Lenguaje principal de todo el sistema



Textual

Interfaz de consola interactiva por pestañas



SQLite

Persistencia del historial con integridad referencial



Git y GitHub

Control de versiones y trabajo colaborativo



VS Code + Remote-SSH

Edición remota del código dentro de la VM



Ubuntu Desktop (VM)

Entorno nativo Linux sobre VirtualBox



subprocess

Ejecución de df, ps, who e ip



fork + threading

Proceso hijo con hilos internos concurrentes

Desarrollo



Cada captura corre en un proceso hijo (os.fork). Dentro de él, dos hilos leen CPU y memoria en paralelo; el resultado viaja al padre por una tubería (pipe) y el padre espera con waitpid antes de continuar, sin bloquear la interfaz.

Resultados

7 / 7

requerimientos funcionales completos

4

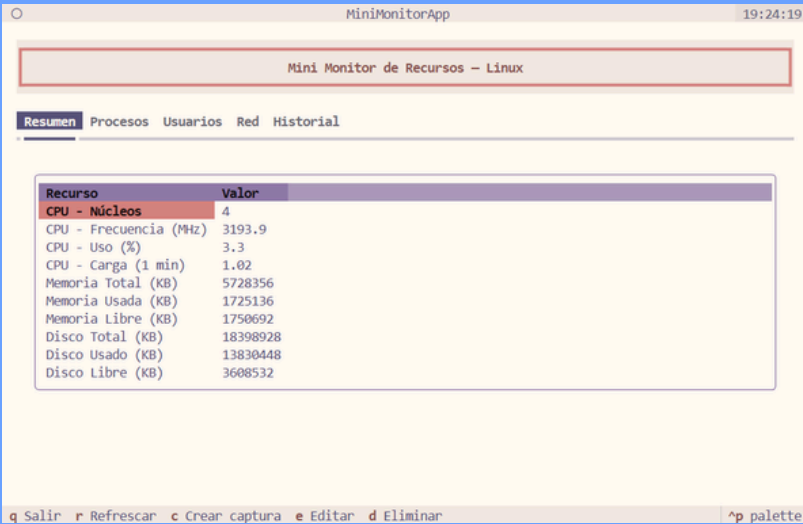
utilidades de Linux validadas (free, df, ps, who)

0

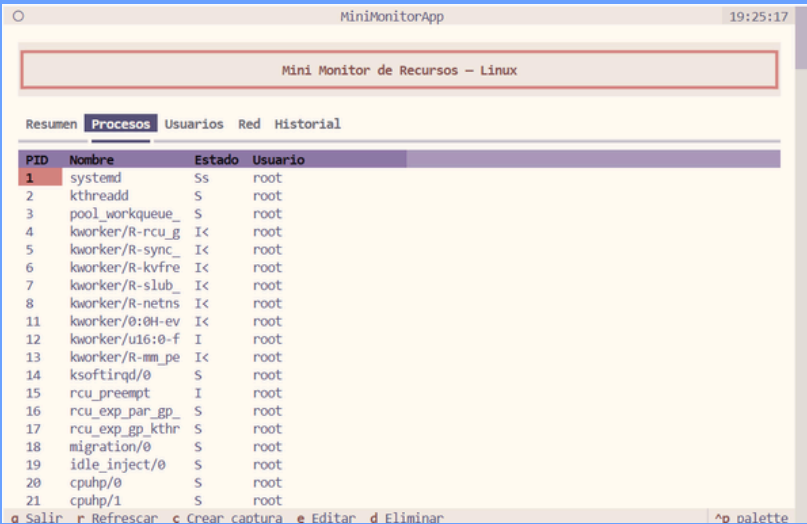
condiciones de carrera detectadas con fork + waitpid

Caso de prueba	Resultado observado
Lectura de CPU/memoria vs. free y top	Valores coincidentes
Lectura de disco vs. df	Valores coincidentes
Procesos y usuarios vs. ps y who	Coincidencia total
Creación de proceso hijo con fork	PID del hijo distinto al del padre
CRUD (crear, consultar, actualizar, eliminar)	Todos los casos validados correctamente

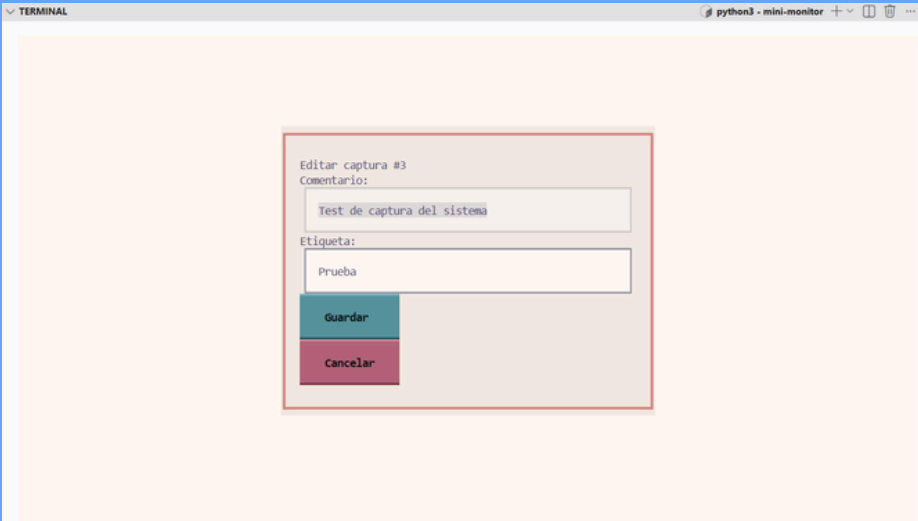
Evidencias



Pestaña Resumen: CPU y memoria leídos desde /proc.



Pestaña Procesos: lista obtenida mediante ps.



Cuadro de diálogo para editar comentario y etiqueta.

```
dnvillagomez@ubuntu-desktop-VirtualBox:~$ cd ProyectoFinal
dnvillagomez@ubuntu-desktop-VirtualBox:~/ProyectoFinal$ cd Mini-Monitor
dnvillagomez@ubuntu-desktop-VirtualBox:~/ProyectoFinal/Mini-Monitor$ sqlite3
db/monitor.db "SELECT id, fecha_hora, comentario, etiqueta FROM monitoreos;"
id|fecha_hora|comentario|etiqueta
1|2026-07-26T13:37:21|Snapshot vía fork+threading|fork_test
2|2026-07-26T20:49:58|Snapshot vía fork+threading|fork_test
dnvillagomez@ubuntu-desktop-VirtualBox:~/ProyectoFinal/Mini-Monitor$
```

Terminal: PID de fork y consulta SQL del snapshot.

Conclusiones

1

Sistema operativo sin cajas negras

La lectura directa de /proc mostró cómo Linux expone su estado interno sin bibliotecas externas.

2

Concurrencia en dos niveles

Combinar fork (proceso) con threading (hilos) permitió comparar en la práctica la concurrencia a nivel de proceso y de hilo.

3

Persistencia con integridad

El historial en SQLite con CRUD y eliminación en cascada reforzó el diseño de esquemas relacionales.

Trabajo futuro: manejo de errores más robusto ante fallos de lectura del sistema y un registro de eventos (logging) para auditar el historial de capturas.

¿Preguntas?

Gracias por su atención

Danny Ayuquina · dmayuquina@espe.edu.ec

Saray Canarte · sacanarte@espe.edu.ec

Domenica Villagomez · dnvillagomez@espe.edu.ec